

The contents of the fib vector can also be printed with the help of the *for()* loop, as shown below:

```
> for(idx in 1:13)
+   print(fib[idx])
```

A sample output is shown below:

```
0
1
1
2
3
5
8
13
21
34
55
89
144
```

is.vector(x) and *is.matrix(x)* can be used to determine whether x is a vector or matrix.

> *is.vector(fib)* will yield the following:

```
[output] TRUE
```

And > *is.matrix(fib)* will yield what follows:

```
[output] FALSE
```

A few more commands (along with the output) related to vector operations are as follows:

```
> min(fib) #for finding minimum value in a vector
[output] 0
> max(fib) #for finding maximum value in a vector
[output] 144
> mean(fib) #for finding mean value
[output] 28.92308
> median(fib) #for finding median value
[output] 8
> var(fib) #for finding variance
[output] 1889.744
> sd(fib) #for finding standard deviation
[output] 43.47118
> sort(fib,decreasing = TRUE) #for sorting in descending order
[output] 144 89 55 34 21 13 8 5 3 2 1 1 0
> sum(fib) #for finding sum of the vector elements
> summary(fib)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.00   2.00   8.00  28.92  34.00  144.00
```

Matrix operations in R

This part of the article dwells on the terseness of complex matrix operations in R.

The command...

```
M <- matrix(c(9,8,7,6,5,4,3,2,1),3,3)
```

...creates a 3 x 3 matrix as follows:

```
      [,1] [,2] [,3]
[1,]    9    6    3
[2,]    8    5    2
[3,]    7    4    1
```



Note: 1) [2,] refers to the second row and [,3] refers to the third column.

2) The numbers are, by default, entered into the matrix in a column-wise fashion.

The command...

```
N <- matrix(c(10,11,12,13,14,15,16,17,18), 3, 3,byrow=TRUE)
```

...creates a 3 x 3 matrix as follows:

```
      [,1] [,2] [,3]
[1,]   10   11   12
[2,]   13   14   15
[3,]   16   17   18
```

In order to display matrix M, the user can simply type the following command...

```
M
```

...at the '>' prompt and press *Ctrl + Enter*.

For adding matrices M and N, and storing the result in matrix A, the user can type the following commands:

```
> A <- M + N
> A
```

The output will appear as follows:

```
      [,1] [,2] [,3]
[1,]   19   17   15
[2,]   21   19   17
[3,]   23   21   19
```

In order to store the transpose of M into TM and for displaying TM, the following commands can be used:

```
> TM <- t(M)
> TM
      [,1] [,2] [,3]
```